



Chương 4. Các quá trình kiểm thử

La Quốc Thắng

1

Nội dung chính

1. Quá trình kiểm thử phần mềm ①
2. Kiểm thử đơn vị ②
3. Kiểm thử tích hợp ③
4. Kiểm thử hệ thống ④

2

Quá trình kiểm thử phần mềm

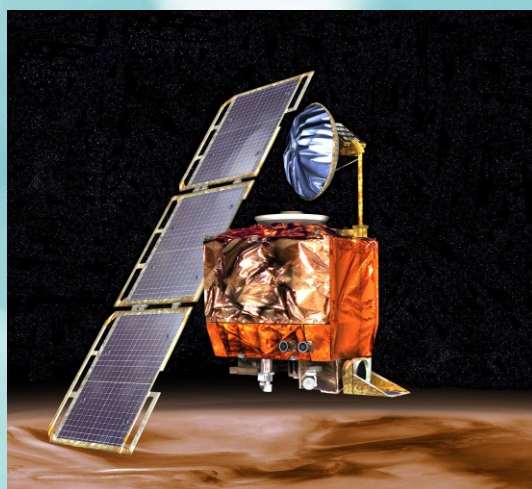
Chương 4. Các quá trình kiểm thử

3

3

Nghiên cứu tình huống

- Mars Climate Orbiter được phóng bởi NASA vào năm 1998, là giai đoạn thứ hai trong chương trình khám phá sao Hỏa.
- Vào năm 1999, tàu Mars Climate Orbiter của NASA biến mất khi đến gần sao Hỏa.



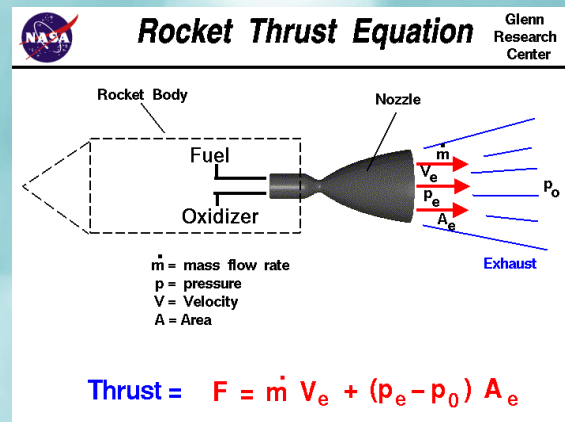
Chương 4. Các quá trình kiểm thử

4

4

Nghiên cứu tình huống

- Nguyên nhân trực tiếp là phần mềm mặt đất sử dụng đơn vị đo lường Anh, trong khi tàu sử dụng hệ mét.
- Các kỹ sư đã không chuyển đổi đơn vị đo của phương trình lực đẩy tên lửa:
 - Từ đơn vị Anh: Pound-force (lb_F)
 - Sang đơn vị SI: Newton ($kg \cdot m/s^2$)
 - $1kg \cdot m/s^2 \approx 0.22481lb_F$



Chương 4. Các quá trình kiểm thử

5

5

Chiến lược kiểm thử

- Quá trình kiểm thử bắt đầu từ module riêng lẻ đến khi tích hợp thành hệ thống hoàn chỉnh.
- Kỹ thuật kiểm thử (như kiểm thử hộp trắng, hộp đen) được chọn tùy theo thời điểm và mục tiêu của từng giai đoạn.
- Kiểm thử có thể được thực hiện bởi lập trình viên hoặc nhóm kiểm thử độc lập.
- Kiểm thử phần mềm giúp phát hiện lỗi; gỡ lỗi (debug) là quá trình phân tích và sửa lỗi. Dù khác nhau, cả hai hoạt động cần được phối hợp trong chiến lược kiểm thử để đảm bảo chất lượng sản phẩm.

Chương 4. Các quá trình kiểm thử

6

6

Sự thích ứng của chiến lược kiểm thử

- Chiến lược kiểm thử thích ứng theo từng mức độ:
 - **Kiểm thử mức thấp (Low-level testing):** Tập trung vào việc xác minh từng đoạn mã nguồn có thực thi đúng theo thiết kế, đảm bảo tính chính xác và ổn định ở cấp độ kỹ thuật nhỏ nhất.
 - **Kiểm thử mức cao (High-level testing):** Hướng đến việc xác minh và thẩm định các chức năng hệ thống chủ yếu, đánh giá tính đúng đắn so với đặc tả kỹ thuật và khả năng đáp ứng yêu cầu từ phía khách hàng.
- Chiến lược kiểm thử cần linh hoạt điều chỉnh theo cấp độ kiểm thử, để đảm bảo chất lượng từ bên trong mã nguồn đến sự vận hành tổng thể của hệ thống.

Yêu cầu đối với một chiến lược kiểm thử hiệu quả

- Mỗi chiến lược kiểm thử phần mềm cần đảm bảo đáp ứng được nhu cầu của các bên liên quan thông qua các yếu tố sau:
 - **Hướng dẫn rõ ràng** cho người thực hiện kiểm thử về cách thức tiến hành kiểm thử đúng quy trình.
 - **Các cột mốc kiểm thử** cho nhà quản lý nhằm kiểm soát tiến độ và đánh giá chất lượng trong từng giai đoạn.
 - **Các tiêu chí đo lường** để phát hiện vấn đề sớm, hỗ trợ ra quyết định kịp thời và giúp khách hàng có thể nhận biết được quá trình kiểm thử đang được thực hiện nghiêm túc.

Tổ chức kiểm thử

- Kiểm thử phần mềm trong hoạt động "Xác minh và thẩm định"
- **Xác minh (Verification):** Là tập hợp các hoạt động nhằm đảm bảo rằng phần mềm thực hiện đúng chức năng kỹ thuật theo đặc tả đã định.
- **Thẩm định (Validation):** Là tập hợp các hoạt động nhằm đảm bảo rằng phần mềm phù hợp với nhu cầu và kỳ vọng của khách hàng.

Câu hỏi	Xác minh	Thẩm định
<i>Ai làm?</i>	Lập trình viên, nhóm kiểm thử kỹ thuật	Nhóm kiểm thử độc lập, khách hàng
<i>Làm cái gì?</i>	Kiểm tra chức năng, logic mã nguồn, tích hợp	Kiểm tra nghiệp vụ, tính năng, trải nghiệm
<i>Khi nào?</i>	Giai đoạn phát triển, tích hợp ban đầu	Giai đoạn kiểm thử hệ thống và chấp nhận
<i>Mối quan hệ?</i>	Xác minh là điều kiện tiền đề cho thẩm định. Cả hai cùng đảm bảo chất lượng phần mềm từ khía cạnh kỹ thuật lẫn giá trị sử dụng.	

Trách nhiệm kiểm thử phần mềm

- Vai trò của kỹ sư phần mềm trong kiểm thử:
 - Các kỹ sư phần mềm là người trực tiếp phát triển sản phẩm, vì thế họ thường tiến hành các kiểm thử ban đầu như kiểm thử đơn vị để phát hiện lỗi sớm.
 - Tuy nhiên, nguyên tắc đảm bảo chất lượng cho thấy rằng việc “người làm sản phẩm trực tiếp kiểm tra sản phẩm” **có thể không khách quan và dễ bỏ sót lỗi.**

Trách nhiệm kiểm thử phần mềm

- Những hiểu lầm phổ biến trong kiểm thử:
 - “*Người phát triển không tham gia kiểm thử*” → Thực tế, họ là người đầu tiên kiểm thử mã nguồn thông qua các kỹ thuật kiểm thử mức thấp.
 - “*Chỉ nên để người ngoài kiểm thử một cách tàn nhẫn*” → Kiểm thử cần đa góc độ, nhưng phải dựa trên quy trình rõ ràng, không nên mang tính đối đầu.
 - “*Người kiểm thử chỉ quan tâm khi bắt đầu kiểm thử*” → Thực tế, người kiểm thử cần tham gia ngay từ giai đoạn phân tích yêu cầu, để hiểu rõ mục tiêu và xây dựng trường hợp kiểm thử phù hợp.

Phân công trách nhiệm kiểm thử

- Phân công kiểm thử theo thực tiễn phát triển phần mềm:
 - Lập trình viên chịu trách nhiệm kiểm thử đơn vị mã nguồn do chính mình phát triển nhằm đảm bảo thực hiện đúng thiết kế (thuộc phạm vi hoạt động xác minh).
 - Lập trình viên cũng có thể tham gia kiểm thử tích hợp, đặc biệt khi các module liên kết cần hiểu rõ logic nội bộ.
 - Nhóm kiểm thử độc lập bắt đầu thực hiện kiểm thử từ giai đoạn mà các cấu trúc phần mềm đã đầy đủ, sẵn sàng cho kiểm thử hệ thống và kiểm thử chức năng nghiệp vụ.

Vai trò của người kiểm thử

- Vai trò phối hợp giữa nhóm kiểm thử độc lập và người phát triển:
 - Nhóm kiểm thử độc lập góp phần loại bỏ những thành kiến như “*người xây dựng không thể kiểm thử sản phẩm tốt*” bằng góc nhìn khách quan.
 - Giúp giảm mâu thuẫn nội bộ, nhất là khi có sự khác biệt giữa nhóm phát triển và nhóm kiểm thử.
 - Đánh giá trung thực nỗ lực phát triển, thông qua việc phát hiện và phân tích lỗi.
 - Cung cấp báo cáo đầy đủ, hỗ trợ tổ chức trong việc kiểm soát và đảm bảo chất lượng phần mềm.

Vai trò của người kiểm thử

- Trách nhiệm của người phát triển phần mềm:
 - Không giao sản phẩm rồi rút lui, mà cần phối hợp kiểm thử liên tục với nhóm kiểm thử.
 - Hợp tác xuyên suốt dự án để đảm bảo quy trình kiểm thử được thực hiện đầy đủ và hiệu quả.

Tiến trình thực hiện kiểm thử

- Trong quá trình phát triển phần mềm, mỗi giai đoạn phát triển đều có giai đoạn kiểm thử đi kèm, nhằm đảm bảo rằng các sản phẩm trung gian và cuối cùng đều đáp ứng yêu cầu kỹ thuật và nghiệp vụ.
- Mô hình chữ V là mô hình phát triển nhấn mạnh sự song song giữa hoạt động phát triển và kiểm thử ở từng giai đoạn.



Chương 4. Các quá trình kiểm thử

15

15

Kiểm thử đơn vị (Unit testing)

Chương 4. Các quá trình kiểm thử

17

17

Kiểm thử đơn vị

- Nội dung kiểm thử:
 - Giao diện.
 - Cấu trúc dữ liệu cục bộ.
 - Đường điều khiển.
 - Điều kiện logic.
 - Phép toán xử lý.
- Câu hỏi:
 - Biến hay module nào cần kiểm thử qua giao diện?
 - Dữ liệu vào/ra nào cần được kiểm tra?
 - Có lỗi xử lý hay logic nào (phép toán, biểu thức)?
 - Có lỗi điều khiển nào (vòng lặp, giá trị biên)?
 - Có lỗi tiềm ẩn nào (ghi chú, mô tả chưa đúng)?

Kiểm thử dữ liệu qua giao diện

- Kiểm thử dòng dữ liệu (data flow testing) qua giao diện nhằm kiểm tra tính nhất quán và chính xác khi module truyền và nhận dữ liệu. Nội dung tập trung vào:
 - **Định lượng:** Xác định số lượng biến hoặc đối tượng dữ liệu được truyền qua giao diện.
 - Ví dụ: Có đúng số lượng biến đầu vào/đầu ra cần thiết không? Có thừa hoặc thiếu dữ liệu không?
 - **Định dạng:** Kiểm tra kiểu dữ liệu và cấu trúc biểu diễn.
 - Ví dụ: Kiểu dữ liệu có phù hợp (int, float, string...)? Định dạng giá trị có tuân thủ quy tắc (yyyy-MM-dd, 2 chữ số thập phân, mã hóa UTF-8,...)?

Kiểm thử dữ liệu qua giao diện

- Các đặc trưng qua giao diện là:
 - Số tham số = số đối số.
 - Tính chất của tham số = tính chất của đối số.
 - Đơn vị của tham số = đơn vị của đối số.
 - Số đối được truyền gọi module = số các tham số đầu vào của module.
 - Thuộc tính các đối được truyền gọi module = thuộc tính của tham số.
 - Đơn vị của đối được truyền để gọi module = đơn vị các tham số module.

Kiểm thử liên quan đến vào/ra (I/O)

- Khi thực hiện I/O cần tiến hành thêm:
 - Tính chất của tập tin có đúng hay không?
 - Các câu lệnh OPEN/CLOSE có đúng không?
 - Đặc tả hình thức đúng với lệnh I/O không?
 - Kích thước của bộ đệm (buffer) có phù hợp với kích thước của bản ghi không?
 - Các tập tin có đang mở trước khi sử dụng không?
 - Các điều kiện kết thúc tập tin (end-of-file) có được xử lý không?
 - Các sai sót thao tác I/O có được xử lý không?
 - Có thông tin văn bản trong đầu ra không?

Kiểm thử cấu trúc dữ liệu cục bộ

- Trong quá trình thiết kế kiểm thử đơn vị, cần lưu ý rằng cấu trúc dữ liệu cục bộ có thể chứa các sai sót nghiêm trọng. Do đó, trường hợp kiểm thử cần được xây dựng để phát hiện các lỗi phổ biến sau:
 - Giá trị ngầm định hoặc giá trị khởi tạo sai.
 - Tên biến không chính xác.
 - Kiểu dữ liệu không nhất quán.
 - Ngoại lệ về địa chỉ, tràn bộ nhớ (overflow).

Kiểm thử xử lý

- Trong kiểm thử đơn vị, cần thiết kế các trường hợp kiểm thử để phát hiện các lỗi liên quan đến thứ tự thực hiện và độ chính xác như:
 - Thứ tự ưu tiên phép tính số học.
 - Sự nhất quán của các phép toán trộn.
 - Khởi tạo/kết thúc không đúng đắn.
 - Độ chính xác của kết quả tính toán.

Kiểm thử điều kiện logic

- Trong quá trình kiểm thử đơn vị, cần phát hiện và đánh giá các lỗi sau:
 - So sánh các kiểu dữ liệu khác nhau.
 - Thứ tự ưu tiên toán tử logic hoặc số học không đúng.
 - Biểu thức so sánh phụ thuộc vào giá trị gần đúng.
 - Giá trị so sánh không hợp lệ hoặc không chính xác.

Kiểm thử dòng điều khiển/biên

- Khi thiết kế trường hợp kiểm thử đơn vị, cần phát hiện các lỗi thường xảy ra ở vòng lặp và tại các giới hạn đầu vào/đầu ra:
 - Lỗi liên quan đến vòng lặp:
 - Vòng lặp không kết thúc hoặc kết thúc sai.
 - Vòng lặp phân kỳ khó kiểm soát.
 - Biến lặp bị thay đổi sai.
 - Lỗi tại các điều kiện biên.

Kiểm thử lỗi tiềm ẩn

- Mô tả sai hoặc mô tả khó hiểu khiến quá trình kiểm thử và xác định lỗi gặp khó khăn.
- Dữ liệu ghi lại không tương ứng với lỗi đã xảy ra, gây trở ngại cho việc phân tích nguyên nhân.
- Điều kiện sai xuất hiện trước khi xảy ra lỗi xử lý, cho thấy logic chưa được kiểm soát chặt chẽ.
- Việc xử lý điều kiện ngoại lệ không đúng đắn, có thể dẫn đến hệ thống hoạt động không ổn định.
- Mô tả sai không cung cấp đủ thông tin cần thiết, làm chậm tiến trình định vị và khắc phục lỗi.

Thủ tục kiểm thử đơn vị

- Sau khi mã nguồn được xây dựng, cần thực hiện rà soát cú pháp và kiểm tra tính đúng đắn trước khi bắt đầu thiết kế trường hợp kiểm thử đơn vị.
- Kiểm thử đơn vị là một bước gắn liền với quá trình lập trình, nhằm đảm bảo từng module hoạt động đúng theo thiết kế.
- Kết quả từ việc rà soát thiết kế đóng vai trò là cơ sở để xây dựng các trường hợp kiểm thử đơn vị, giúp phát hiện các loại lỗi cụ thể đã được nhận diện trước đó.
- Mỗi trường hợp kiểm thử đơn vị cần xác định rõ tập kết quả mong đợi để so sánh với kết quả thực thi và phát hiện sai lệch.



Kỹ thuật kiểm thử đơn vị

- Vì module phần mềm không hoạt động một cách độc lập, nên khi kiểm thử đơn vị, cần xây dựng phần mềm hỗ trợ kiểm thử gồm bộ lái và/hoặc cuống.
- **Bộ lái (Driver):** Là một chương trình đơn giản, thường đóng vai trò như hàm main(), dùng để:
 - Khởi động module cần kiểm thử.
 - Cung cấp dữ liệu đầu vào.
 - Thu nhận và kiểm tra kết quả đầu ra.
- **Cuống (Stub):** Là các chương trình giả lập, thay thế cho các module phụ thuộc (thứ cấp) mà module đang kiểm thử có thể gọi tới. Cuống giúp:
 - Mô phỏng hành vi của các module chưa phát triển.
 - Đảm bảo việc kiểm thử module chính không bị gián đoạn do thiếu thành phần phụ.

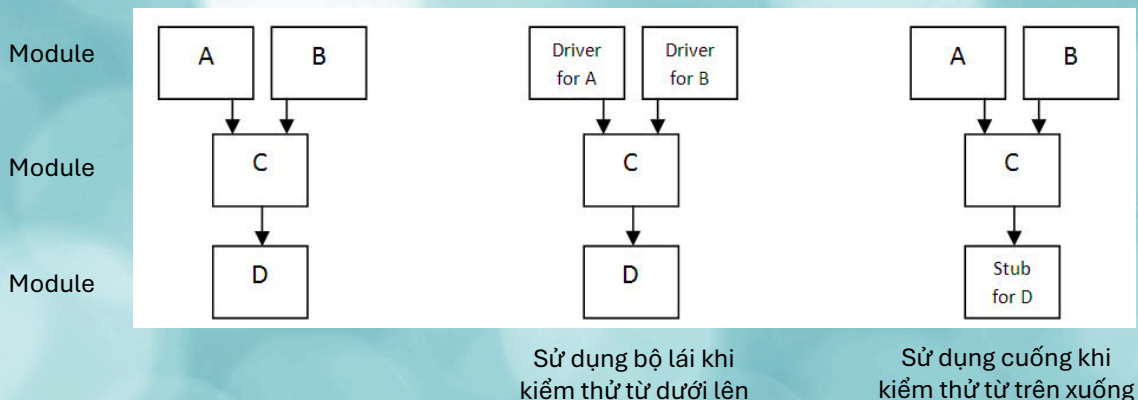
Chương 4. Các quá trình kiểm thử

28

28

Kỹ thuật kiểm thử đơn vị

- Một cuống mô phỏng hành vi của thành phần được gọi, trong khi trình bộ lái mô phỏng hành vi của thành phần đang gọi.

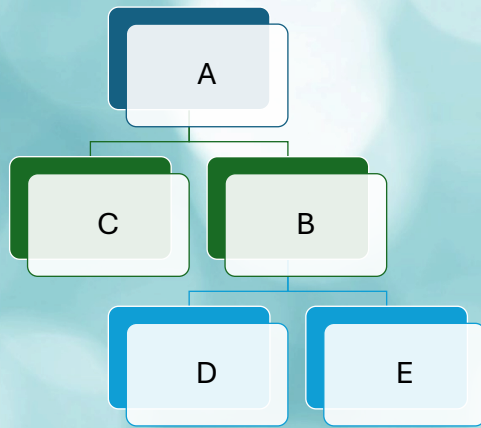


Chương 4. Các quá trình kiểm thử

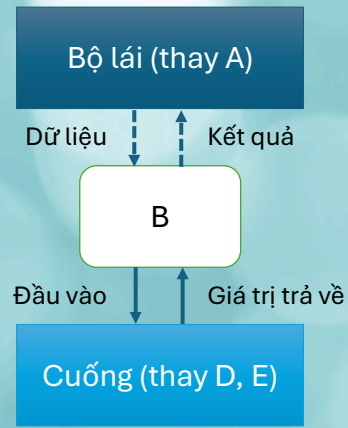
29

29

Kỹ thuật kiểm thử đơn vị



Sơ đồ hệ thống



Kiểm thử module B

Chương 4. Các quá trình kiểm thử

30

30

Kỹ thuật kiểm thử đơn vị

- Cuồng hay chương trình giả lập sử dụng giao diện của module thứ cấp để thực hiện các thao tác tối thiểu nhằm kiểm chứng đầu vào và giá trị trả về của module chính.
- Bộ lái và cuồng không được phân phối kèm sản phẩm cuối cùng; chúng chỉ phục vụ mục tiêu kiểm thử, nên được xem là chi phí hành chính.
- Nếu thiết kế đơn giản, chi phí này tương đối thấp. Tuy nhiên, trong nhiều trường hợp module có tính phức tạp cao, bộ lái và cuồng cần thiết kế công phu — **làm tăng chi phí kiểm thử đơn vị**.
- Chính vì vậy, việc kiểm thử đầy đủ ở cấp đơn vị đôi khi bị trì hoãn và được thực hiện đầy đủ hơn ở giai đoạn kiểm thử tích hợp, khi các module phụ thuộc đã hoàn thiện.

Chương 4. Các quá trình kiểm thử

31

31

Một số thư viện kiểm thử đơn vị

1. JUnit



6. PyUnit (unittest)



2. Jbehave



7. Cypress



3. NUnit



4. XUnit



5. Robot



Chương 4. Các quá trình kiểm thử

32

32

JUnit trong Java

```
package com.vogella.junit5;

public class Calculator {

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

```
package com.vogella.junit5;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.Test;

class CalculatorTest {

    Calculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator();
    }

    @Test
    @DisplayName("Simple multiplication should work")
    void testMultiply() {
        assertEquals(20, calculator.multiply(4, 5),
            "Regular multiplication should work");
    }

    @RepeatedTest(5)
    @DisplayName("Ensure correct handling of zero")
    void testMultiplyWithZero() {
        assertEquals(0, calculator.multiply(0, 5), "Multiple with zero should be zero");
        assertEquals(0, calculator.multiply(5, 0), "Multiple with zero should be zero");
    }
}
```

Chương 4. Các quá trình kiểm thử

33

33

NUnit trong C#

```

1 public class Calculator
2 {
3     public int Add(int x, int y)
4     {
5         return x + y;
6     }
7
8     public int Subtract(int x, int y)
9     {
10        return x - y;
11    }
12 }
13

```

```

1 using NUnit.Framework;
2
3 [TestFixture]
4 public class CalculatorTest
5 {
6     private Calculator _cal;
7
8     [SetUp]
9     public void Setup()
10    {
11        _cal = new Calculator();
12    }
13
14    [Test]
15    public void OnePlusOneEqualTwo()
16    {
17        Assert.AreEqual(2, _cal.Add(1, 1));
18    }
19
20    [Test]
21    public void TwoPlusTwoEqualFour()
22    {
23        Assert.AreEqual(4, _cal.Add(2, 2));
24    }
25
26    [Test]
27    public void FourPlusOneEqualFive()
28    {
29        Assert.AreEqual(5, _cal.Add(4, 1));
30    }
31 }

```

Chương 4. Các quá trình kiểm thử

34

34

Kiểm thử tích hợp (Intergration testing)

Chương 4. Các quá trình kiểm thử

36

36

Kiểm thử tích hợp

- Kiểm thử tích hợp (Integration testing) là quá trình kiểm tra sự phối hợp giữa các module đã được kiểm thử đơn vị nhằm đảm bảo các bộ phận chức năng hoặc hệ thống con hoạt động đúng theo thiết kế.
- Đây là một kỹ thuật có tính hệ thống, được sử dụng để xây dựng cấu trúc chương trình hoàn chỉnh từ các module riêng lẻ, theo đúng kiến trúc đã định.
- Phương pháp tích hợp module:
 - Từ trên xuống (Top-down).
 - Từ dưới lên (Bottom-up).
 - Kẹp (Sandwich).
- Tích hợp đồng thời (Big Bang).

37

Kiểm thử tích hợp

Chiến lược	Mô tả ngắn gọn	Ưu điểm	Nhược điểm
Từ trên xuống	Tích hợp từ module cấp cao xuống module cấp thấp	Kiểm thử luồng điều khiển sớm	Cần cuống cho module thấp
Từ dưới lên	Tích hợp từ module cấp thấp lên module cấp cao	Dễ phát triển cuống	Chưa kiểm thử luồng tổng thể
Kẹp	Kết hợp từ trên xuống và từ dưới lên đồng thời	Cân bằng kiểm thử cả hai chiều	Quản lý tích hợp phức tạp
Big Bang	Tích hợp toàn bộ module một lần, rồi kiểm thử toàn thể	Đỡ tốn thời gian lập trình	Khó xác định vị trí lỗi

38

Các lỗi thường gặp trong kiểm thử tích hợp

- Dữ liệu bị mất hoặc sai lệch khi truyền qua giao diện giữa các module, do không tương thích về kiểu, định dạng hoặc đơn vị đo.
- Hiệu ứng không mong muốn từ một module gây ra cho module khác, dẫn tới hành vi bất ổn hoặc kết quả sai trong toàn bộ hệ thống con.
- Kết hợp các chức năng phụ nhưng không tạo ra được chức năng chính mong đợi, thường do thiết kế tích hợp không đúng hoặc sai logic phối hợp.
- Sai sót riêng rẽ ở các module bị phóng đại khi chúng tương tác, dẫn đến lỗi toàn cục vượt quá mức có thể chấp nhận được.
- Cấu trúc dữ liệu toàn cục lộ ra các vấn đề thiết kế như chia sẻ không an toàn, xung đột truy cập hoặc trạng thái bị phụ thuộc — vốn không thể phát hiện khi kiểm thử đơn vị.

Nguy cơ khi kiểm thử tích hợp kiểu “Big Bang”

- Khi kết hợp tất cả module đã kiểm thử đơn vị thành một chương trình hoàn chỉnh và tiến hành kiểm thử toàn bộ, dễ dẫn đến kết quả rối loạn, khó kiểm soát.
- Các lỗi phát sinh thường không thể xác định nguyên nhân chính xác, vì các module tương tác đồng thời, gây ra những hiệu ứng chéo và phụ thuộc phức tạp.
- Quá trình sửa lỗi trở nên khó khăn và triền miên:
 - Sửa một lỗi có thể kích hoạt lỗi khác.
 - Mỗi lỗi mới lại làm mờ đi dấu vết nguyên nhân gốc.
 - Việc cô lập và xử lý lỗi trở thành một vòng luẩn quẩn không hồi kết.

Chiến lược tích hợp từ trên xuống

- Chiến lược tích hợp từ trên xuống (Top-down integration) là phương pháp xây dựng và kiểm thử chương trình bắt đầu từ module điều khiển chính (“main”).
- Các module phụ trợ được gắn dần vào theo dòng điều khiển logic, nhằm đảm bảo cấu trúc chương trình được kiểm soát và phát triển có hệ thống.
- Trong quá trình kiểm thử, nếu các module cấp dưới chưa hoàn thành, chúng được sử dụng để giả lập hành vi của chúng.

Chiến lược tích hợp từ trên xuống

- Các kiểu triển khai tích hợp từ trên xuống:
 - **Chiều “sâu trước” (Depth-first):** Tích hợp theo nhánh chức năng dài nhất trước.
 - **Chiều “rộng trước” (Breadth-first):** Tích hợp các module ở cùng một tầng độ sâu trước.
 - **Kiểu hỗn hợp:** Kết hợp cả hai phương pháp tùy theo độ ưu tiên chức năng hoặc độ phức tạp logic của hệ thống.

Tiến trình tích hợp từ trên xuống

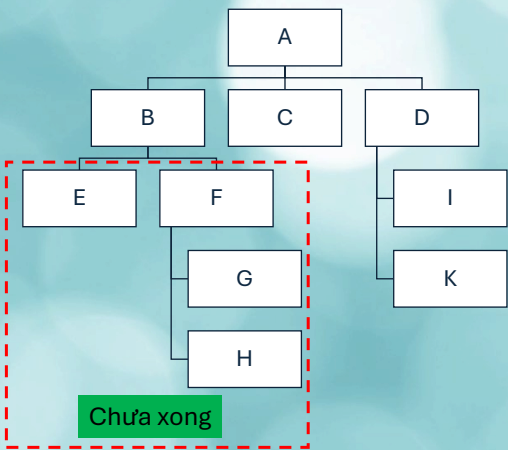
1. Bắt đầu với module điều khiển chính, được sử dụng như một bộ lái kiểm thử để khởi động chương trình.
2. Các module phụ trợ trực tiếp của module điều khiển được thay thế bằng các cuống tạm thời.
3. Lần lượt thay thế từng cuống bằng module thực tương ứng, rồi tiến hành kiểm thử ngay khi tích hợp module mới.
4. Khi hoàn tất kiểm thử module vừa tích hợp, chuyển sang thay thế cuống tiếp theo, tiếp tục quy trình kiểm thử — lặp lại từ bước tích hợp đến kiểm thử.
5. Có thể thực hiện kiểm thử hồi quy sau mỗi lần thay thế cuống bằng module thật để đảm bảo không xuất hiện lỗi mới do tích hợp.
6. Quá trình tiếp tục cho đến khi toàn bộ cấu trúc chương trình được xây dựng đầy đủ từ module chính tới các module phụ thuộc cuối cùng.

Chương 4. Các quá trình kiểm thử

43

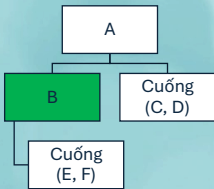
43

Sơ đồ tích hợp từ trên xuống

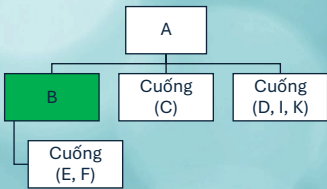


Hệ cần kiểm thử

Kiểm thử A



Kết hợp theo chiều rộng



Kết hợp theo chiều sâu

Chương 4. Các quá trình kiểm thử

44

44

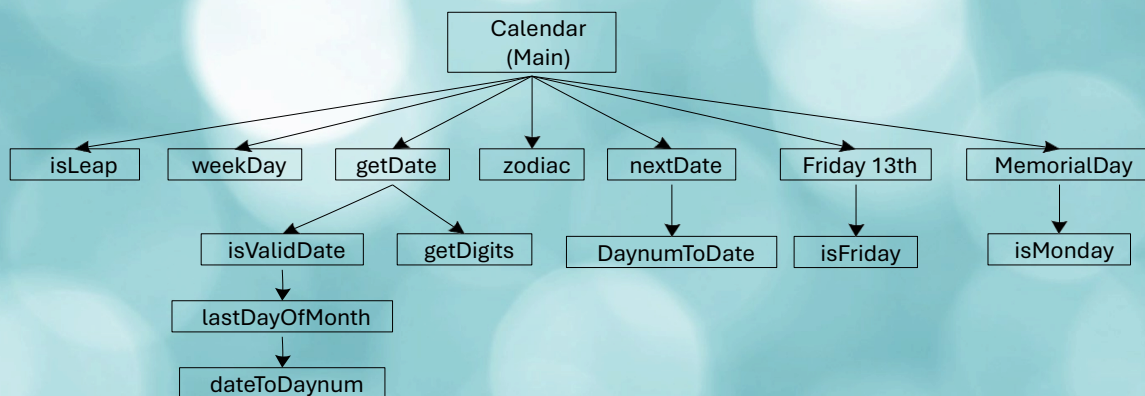
Ví dụ: Chương trình lịch

- Ngày ở dạng mm, dd, yyyy
- Các chức năng của lịch:
 - Xem ngày của ngày hôm sau (nextDate).
 - Xem ngày bất kỳ.
 - Xem cung hoàng đạo.
 - ...

45

Ví dụ: Chương trình lịch

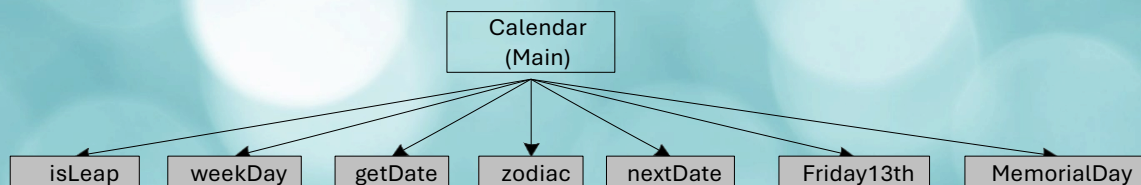
- Phân rã chức năng của chương trình:



46

Ví dụ: Chương trình lịch

- Các module màu xám là các cuống trả về các giá trị chính xác khi được tham chiếu:



Chương 4. Các quá trình kiểm thử

47

47

Ví dụ: Chương trình lịch

- Tạo các cuống cho từng module, chẳng hạn weekDayStub:

```

Procedure weekDayStub(mm, dd, yyyy, dayName):
  If ((mm = 10) AND (dd = 28) AND (yyyy = 2013))
    Then dayName = "Monday"
  EndIf
  ...
  If ((mm = 10) AND (dd = 30) AND (yyyy = 2013))
    Then dayName = "Wednesday"
  EndIf
  
```

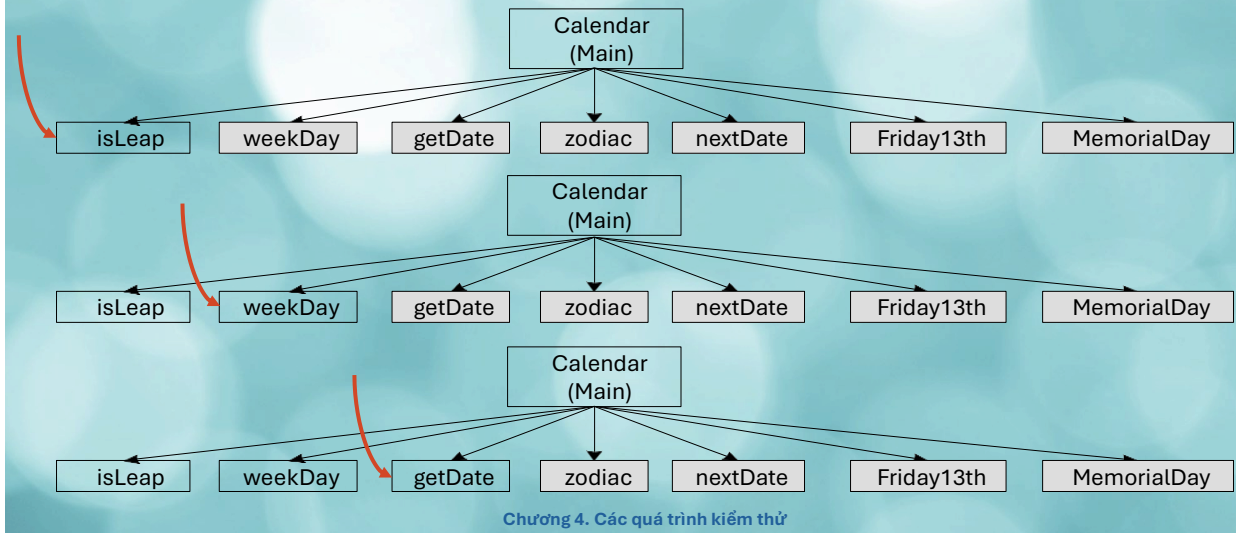
Chương 4. Các quá trình kiểm thử

48

48

Ví dụ: Chương trình lịch

- Tại một thời điểm thì thay thế một cuống:



49

49

Nhận xét chiến lược tích hợp từ trên xuống

- Hạn chế logic trong tích hợp từ trên xuống:
 - Tuy không phức tạp về mặt tổ chức, phương pháp tích hợp từ trên xuống vẫn tiềm ẩn vấn đề logic: khi xử lý ở module cấp thấp thì cần phải có sự tương xứng với module cấp cao, nhưng cuống thường không đủ khả năng đáp ứng yêu cầu này.
 - Khi sử dụng cuống để thay thế module thấp, sẽ không có dữ liệu có ý nghĩa nào được truyền ngược lên module cấp cao — gây ra việc kiểm thử không đầy đủ hoặc không phản ánh đúng luồng xử lý thực tế.

Chương 4. Các quá trình kiểm thử

50

50

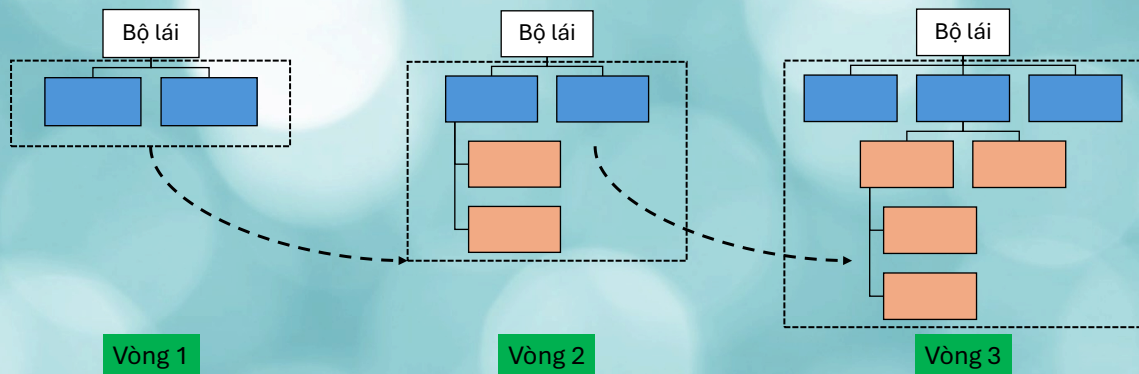
Nhận xét chiến lược tích hợp từ trên xuống

- Người kiểm thử có thể chọn một trong ba cách ứng phó sau:
 - Trì hoãn một số kiểm thử cho đến khi cuống được thay thế bởi module thực, nhằm đảm bảo kiểm thử có dữ liệu thực và logic đầy đủ.
 - Phát triển các cuống thông minh hơn – tức là xây dựng cuống có thể xử lý một phần chức năng module dưới, giúp kiểm thử không bị nghèo nàn về dữ liệu.
 - Kết hợp tích hợp từ dưới lên, bổ sung module cấp thấp trước để có dữ liệu thực – từ đó cải thiện tính đầy đủ và chính xác khi kiểm thử module điều khiển.

Chiến lược tích hợp từ dưới lên

- Bắt đầu từ các module cấp thấp nhất, thường là các hàm xử lý cơ bản không phụ thuộc vào module nào khác.
- Các module này được gom thành các cụm (cluster), mỗi cụm thực hiện một chức năng phụ trợ độc lập trong hệ thống.
- Viết bộ lái (driver) cho từng cụm để cung cấp đầu vào cho các module trong cụm, thu nhận và kiểm tra đầu ra của cụm trong quá trình kiểm thử.
- Sau khi cụm đó được kiểm thử thành công, tháo bỏ driver, rồi tích hợp cụm vào cấp trên trong cấu trúc chương trình.
- Tiếp tục thực hiện quá trình này một cách ngược lên, cho đến khi toàn bộ hệ thống được lắp ghép đầy đủ và kiểm thử hoàn chỉnh.

Sơ đồ tích hợp từ dưới lên



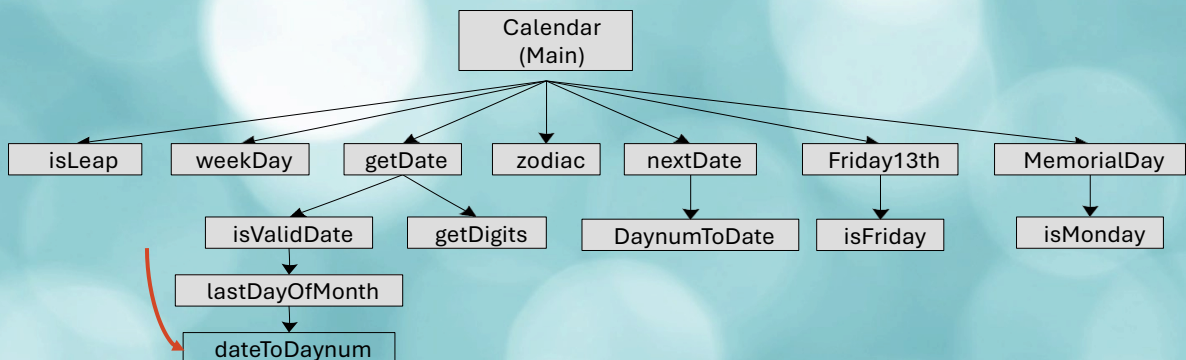
Chương 4. Các quá trình kiểm thử

53

53

Ví dụ: Chương trình lịch

- Điểm bắt đầu của tích hợp từ dưới lên:



Chương 4. Các quá trình kiểm thử

54

54

Nhận xét kiểm thử tích hợp

Tiêu chí	Từ trên xuống (Top-down)	Từ dưới lên (Bottom-up)
Ưu điểm	Có ngay chức năng điều khiển hệ thống	Không cần cuống, trường hợp kiểm thử dễ thiết kế
Nhược điểm	Cần xây dựng cuống, khó khăn khi cuống không đủ logic	Chưa có chương trình chỉnh thể trong giai đoạn đầu
Vấn đề đi kèm	Cuống không truyền dữ liệu có ý nghĩa lên module cấp cao	Cần driver để kiểm thử từng cụm trước khi tích hợp vào hệ thống
Xử lý của người kiểm thử	- Trì hoãn kiểm thử đến khi có module thật - Xây dựng cuống thông minh hơn - Kết hợp tích hợp từ dưới lên	Không cần cuống, chủ động xây dựng cụm và kiểm thử với bộ lái trước

Kiểm thử tích hợp hỗn hợp

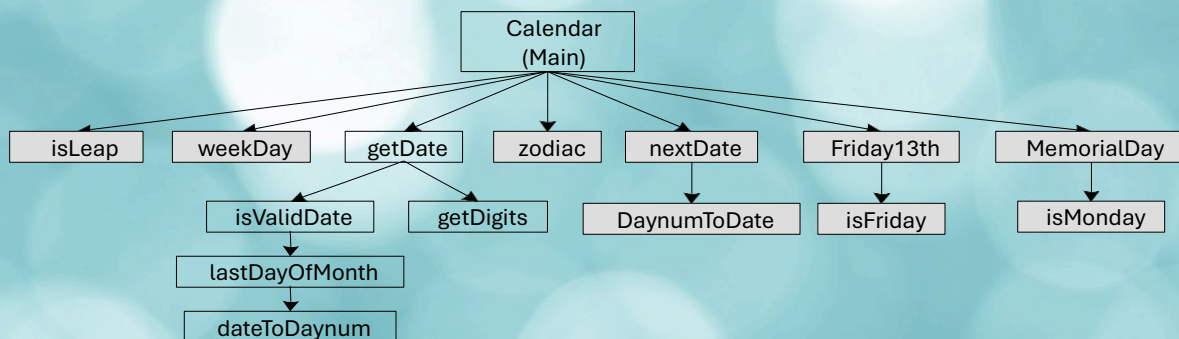
- Chiến lược tích hợp hỗn hợp (Sandwich):
 - Các module quyết định (logic modules)** thường chứa luồng điều khiển, điều phối hoạt động chương trình — do đó cần được tích hợp và kiểm thử theo phương pháp top-down để phát hiện sớm lỗi thiết kế hoặc sai lệch kiến trúc.
 - Các module chức năng (operational modules)** đóng vai trò xử lý dữ liệu, thực hiện chức năng cụ thể — vì vậy thường được tích hợp theo phương pháp bottom-up nhằm:
 - Kiểm thử khả năng tái sử dụng của từng module.
 - Giảm bớt số lượng cuống do sử dụng module thực thay thế.

Kiểm thử tích hợp hỗn hợp

- Việc kết hợp cả hai phương pháp giúp tận dụng được lợi thế của từng hướng tiếp cận:
 - Từ trên xuống** hỗ trợ kiểm soát logic tổng thể và phát hiện lỗi sớm.
 - Từ dưới lên** hỗ trợ kiểm thử đầy đủ module chức năng, giảm chi phí giả lập.

57

Ví dụ: Chương trình lịch



58

Kiểm thử hồi quy

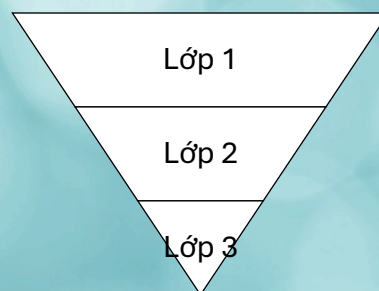
- Mỗi khi tích hợp module mới hoặc sửa một lỗi (bug), phần mềm sẽ thay đổi — và chính sự thay đổi này có thể gây ra lỗi ngoài ý muốn ở các chức năng đã hoạt động đúng trước đó.
- Kiểm thử hồi quy (Regression testing) là quá trình chạy lại một số kiểm thử trước đó để đảm bảo rằng các chỉnh sửa không tạo ra hiệu ứng phụ tiêu cực.

Kiểm thử hồi quy

- Cách thực hiện kiểm thử hồi quy:
 - **Chạy lại một phần bộ trường hợp kiểm thử:** chỉ chọn các kiểm thử có liên quan trực tiếp đến phần đã thay đổi hoặc có thể bị ảnh hưởng.
 - **Sử dụng công cụ kiểm thử tự động:** giúp chạy lại nhanh chóng, chính xác và tiết kiệm thời gian — đặc biệt hiệu quả khi phần mềm có quy mô lớn hoặc tần suất thay đổi cao.

Các lớp kiểm thử hồi quy

- Khi thực hiện kiểm thử hồi quy, cần xây dựng bộ test-case có tổ chức theo 3 lớp:
 - Lớp toàn bộ hệ thống:** Kiểm tra lại tất cả các chức năng của phần mềm để đảm bảo hoạt động tổng thể vẫn đúng sau khi có thay đổi.
 - Lớp chức năng bị ảnh hưởng:** Tập trung kiểm thử các chức năng liên quan trực tiếp đến phần thay đổi — giúp kiểm tra vùng tác động lan truyền.
 - Lớp thành phần bị thay đổi:** Kiểm thử các module, hàm hoặc đoạn mã cụ thể đã bị chỉnh sửa — xác minh sửa lỗi không gây ra lỗi mới tại chỗ.



Quản lý khối lượng kiểm thử hồi quy

- Việc bổ sung trường hợp kiểm thử sau mỗi thay đổi có thể khiến **số lượng kiểm thử tăng nhanh**, gây quá tải và ảnh hưởng đến tiến độ dự án.
- Vì vậy, các trường hợp kiểm thử hồi quy nên được **lựa chọn có trọng tâm**, chỉ tập trung vào nhóm chức năng hoặc lỗi có mức độ ưu tiên cao.
- Không nên kiểm thử lại toàn bộ** phần mềm sau mỗi thay đổi nhỏ, vì vừa không khả thi, vừa không hiệu quả về chi phí và thời gian.

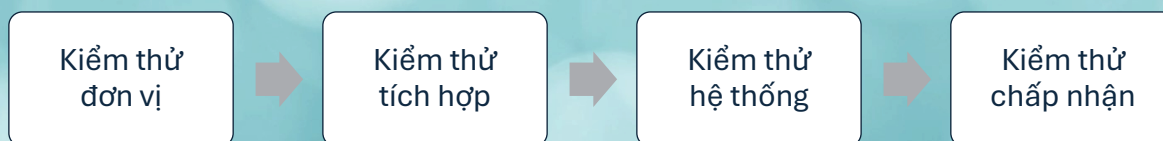
Nhận xét

- Người kiểm thử cần nhận diện các **module tới hạn (critical modules)** — tức những module cần ưu tiên kiểm thử sớm do đặc tính dễ phát sinh lỗi hoặc có vai trò trọng yếu trong hệ thống.
- Tập trung xử lý một hoặc vài yêu cầu cốt lõi của phần mềm, có ảnh hưởng trực tiếp đến chức năng chính.
- Có mức điều khiển cao trong cấu trúc chương trình, ví dụ như module điều phối luồng dữ liệu hoặc gọi nhiều module khác.
- Phức tạp hoặc dễ sai, có thể được xác định thông qua độ phức tạp vòng lặp, số lượng nhánh hoặc độ rối logic.
- Có yêu cầu thực hiện rõ ràng, định lượng được, nên có thể thiết kế kiểm thử có mục tiêu cụ thể.

Kiểm thử hệ thống

Kiểm thử hệ thống

- Kiểm thử hệ thống (System testing) là quá trình kiểm thử toàn diện phần mềm sau khi hoàn thiện tích hợp.
- Mục tiêu là đảm bảo mọi chức năng hoạt động chính xác trong môi trường đã định trước, mô phỏng sát với điều kiện thực tế triển khai.



Kiểm thử hệ thống

- Các vùng trọng tâm trong kiểm thử hệ thống:
 - Kiểm thử chức năng và hiệu suất hệ thống.
 - Kiểm thử độ tin cậy và hồi phục (Recovery Test).
 - Kiểm thử cài đặt hệ thống (Installation Test).
 - Kiểm thử điều kiện đặc biệt (Stress & Load Test).
 - Kiểm thử chấp nhận người dùng (Acceptance/Alpha Test).
 - Tích hợp phần cứng và phần mềm.
 - Tích hợp phần mềm bên ngoài (3rd-party).

Kiểm thử hệ thống

- Kiểm thử hồi phục (Recovery testing):
 - Mục tiêu: Đánh giá khả năng phục hồi của hệ thống khi gặp sự cố, như lỗi phần mềm, mất kết nối, ngắt điện, lỗi thiết bị,...
 - Trọng tâm kiểm thử:
 - Kiểm tra xem hệ thống có thể khôi phục trạng thái hoạt động bình thường hay không sau khi xảy ra lỗi.
 - Đặc biệt quan tâm đến khả năng bảo toàn và phục hồi dữ liệu.
 - Đo thời gian khôi phục trong các tình huống khác nhau.

Kiểm thử hệ thống

- Kiểm thử khả năng chịu đựng (Stress testing):
 - Mục tiêu: Đánh giá xem hệ thống có duy trì hoạt động ổn định khi bị đặt vào điều kiện tải nặng hoặc gián đoạn liên tục — vượt khỏi mức hoạt động thông thường.
 - Trọng tâm kiểm thử:
 - Kiểm tra xử lý đồng thời.
 - Đặt hệ thống vào điều kiện bất thường.
 - Kiểm tra mức giới hạn.
 - Kiểm tra độ nhạy (Sensitivity Test).
 - Phát hiện tổ hợp đầu vào hợp lệ nhưng nguy hiểm.

Kiểm thử hệ thống

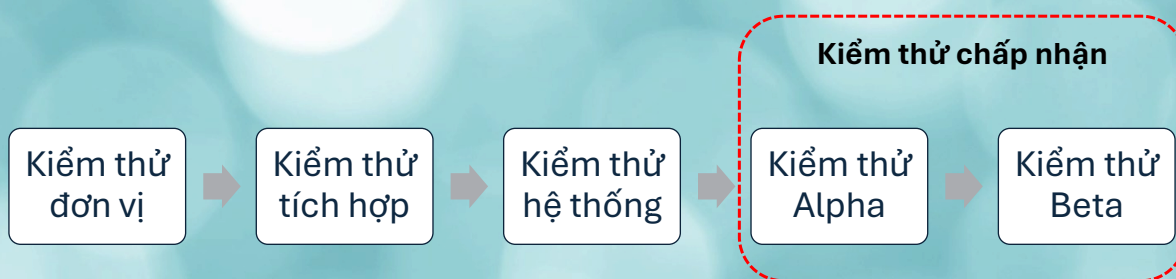
- Kiểm thử độ an toàn (Security testing):
 - Mục tiêu: Đánh giá khả năng của hệ thống bảo vệ dữ liệu và ngăn chặn truy cập trái phép, đảm bảo tính bảo mật toàn diện khi triển khai thực tế.
 - Trọng tâm kiểm thử:
 - Kiểm tra cơ chế chống truy cập không hợp lệ.
 - Đánh giá khả năng chống lại tấn công từ bên ngoài.
 - Phân tích chi phí thâm nhập so với giá trị dữ liệu.

Kiểm thử hệ thống

- Kiểm thử hiệu suất (Performance testing):
 - Mục tiêu: Đánh giá hiệu năng thời gian chạy của phần mềm nhằm xác minh khả năng đáp ứng đúng hạn và ổn định trong môi trường vận hành thực tế.
 - Trọng tâm kiểm thử:
 - Đo tốc độ xử lý và phản hồi.
 - Theo dõi mức sử dụng tài nguyên.
 - Kết hợp kiểm thử khả năng chịu đựng (Stress Testing).
 - Yêu cầu phối hợp cả phần mềm và phần cứng.
 - Xuất hiện xuyên suốt toàn bộ quá trình thử nghiệm.

Kiểm thử Alpha và Beta

- Kiểm thử Alpha và Beta được áp dụng khi hệ thống đã gần hoàn thiện và bắt đầu được đưa vào sử dụng bởi nhiều người dùng thực tế.



Kiểm thử Alpha

- Thực hiện nội bộ bởi nhóm phát triển hoặc trong môi trường giả lập gần giống thực tế.
- Mục tiêu là phát hiện các lỗi khó nhận biết trong quá trình phát triển, đặc biệt những lỗi liên quan đến hành vi người dùng.
- Người kiểm thử có thể là nhân viên công ty hoặc người dùng được chọn lọc kỹ.

Kiểm thử Beta

- Triển khai phần mềm đến nhóm người dùng cuối thực sự, thường là khách hàng tiềm năng trong môi trường sử dụng thực tế.
- Giúp phát hiện những lỗi mà đội phát triển không lường trước được — chẳng hạn như xung đột cấu hình, hành vi không mong muốn khi sử dụng lâu dài...
- Phản hồi từ người dùng giúp hoàn thiện sản phẩm trước khi chính thức phát hành.

Ví dụ

Quy trình kiểm thử Chrome

Phiên bản Chrome	Logo	Mục tiêu kiểm thử	Thời gian cập nhật	Đối tượng sử dụng	Tính ổn định
Canary (Alpha)		Phát hiện lỗi sớm	Hằng ngày	Nhà phát triển	Rất thấp
Dev		Kiểm thử bản gần hoàn chỉnh	1-2 lần/tuần	Nhà phát triển	Trung bình - thấp
Beta		Kiểm thử thực tế	Hằng tuần, bản lớn 4 tuần/lần	Người dùng thử nghiệm	Trung bình - cao
Stable		Sử dụng chính thức	Cập nhật nhỏ 2-3 tuần/lần, cập nhật lớn 4 tuần/lần	Người dùng phổ thông	Cao

Kiểm thử chấp nhận

- Kiểm thử chấp nhận (Acceptance testing):
 - Mục tiêu: Xác nhận rằng phần mềm đáp ứng đầy đủ yêu cầu thực tế của khách hàng hoặc người dùng cuối, trước khi chính thức triển khai sử dụng.
 - Đặc điểm kiểm thử chấp nhận:
 - Thực hiện sau kiểm thử hệ thống, đặc biệt khi phần mềm được phát triển riêng cho khách hàng cụ thể.
 - Khách hàng hoặc người dùng cuối trực tiếp thực hiện kiểm thử, để đánh giá sản phẩm từ góc độ sử dụng thực tế.

Kiểm thử chấp nhận

- Yêu cầu thực hiện:
 - Dựa trên tài liệu yêu cầu phần mềm do khách hàng cung cấp hoặc được ký kết giữa các bên liên quan.
 - Hướng dẫn sử dụng được dùng để hỗ trợ người kiểm thử tiến hành thao tác chính xác.
 - Có thể tái sử dụng các trường hợp kiểm thử từ kiểm thử hệ thống để đảm bảo bao phủ đầy đủ chức năng chính.
 - Phần mềm phải được chạy trong môi trường thực tế — bao gồm cả phần cứng, phần mềm và dữ liệu sản xuất (nếu có).
 - Khách hàng sẽ là người đưa ra quyết định cuối cùng: phần mềm có đạt yêu cầu hay không và có thể chấp nhận bàn giao hay cần chỉnh sửa thêm.

Tài liệu tham khảo

1. Bài giảng Kiểm thử phần mềm, Đại học Bách khoa Hà Nội
2. <https://www.frugaltesting.com/blog/understanding-bugs-defects-errors-faults-and-failures-in-software-testing>
3. <https://cs.gmu.edu/~offutt/softwaretest/>
4. <https://malenezi.github.io/malenezi/SE401/>